# SIMATS ENGINEERING

## ASSESSMENT TOOL 3

**COURSE NAME: OPERATING SYSTEMS COURSE CODE: CSA04**

---

**COURSE OUTCOME COVERED**

***CO2:*** Analyze process management and deadlock handling techniques to improve CPU utilization and system performance(BL3-BL4)

*Assessment Tool Weightage: 30%*

---

**QUESTIONS: 10** **Total Marks:50**

Annexure A

**Debugging or Optimization task**

---

***Code of Conduct:***

*I* **Shaik Rafiqhuddin (192525129)** *(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.*

*I understand that any violation of academic integrity rules will result in disciplinary action.*

***Signature of the student:*** 

# QUESTION 1

*Deadlock Debugging — Code Trace*

**10 MARKS**

## Original Question

**Q1. Deadlock Debugging (Code Trace)**

You are given the following pseudo-code for two processes sharing resources R1 and R2:

Process P1: lock(R1) → wait(100 ms) → lock(R2) → critical section → unlock(R2) → unlock(R1)

Process P2: lock(R2) → wait(100 ms) → lock(R1) → critical section → unlock(R1) → unlock(R2)

Identify the exact condition that causes deadlock. Suggest an optimisation to eliminate the deadlock without changing process logic.

*(10 Marks)*

## ANSWER

### Problem Statement

Two processes acquire the same two resources in opposite order, and each holds one resource while waiting 100 ms for the other — identify why this deadlocks and fix it without altering what each process actually does in its critical section.

### Given (Buggy) Pseudocode

```
Process P1:                    Process P2:
  lock(R1)                       lock(R2)
  wait(100 ms)                   wait(100 ms)
  lock(R2)                       lock(R1)
  critical section               critical section
  unlock(R2)                     unlock(R1)
  unlock(R1)                     unlock(R2)
```

### Root Cause / Exact Deadlock Condition

Both processes start at roughly the same time. P1 locks R1 and P2 locks R2 essentially simultaneously; each then waits 100 ms (long enough for the other to also grab its first lock) before requesting the second resource. When they wake, P1 requests R2 (held by P2) and P2 requests R1 (held by P1) — each holds one resource and waits indefinitely for the other. This is a textbook circular wait: P1 → (waits for) → P2 → (waits for) → P1.

### Theory — Coffman's Conditions

All four necessary conditions for deadlock are present simultaneously: (1) Mutual Exclusion — R1/R2 are exclusively lockable; (2) Hold-and-Wait — each process holds its first lock while waiting for the second; (3) No Preemption — locks are only released voluntarily via unlock(); (4) Circular

Wait — P1→R2→P2→R1→P1 forms a cycle. Removing any one of these breaks the deadlock; the standard, least invasive fix targets Circular Wait.

### Resource Allocation Graph (at the moment of deadlock)

| Process | Relationship | Resource | Edge Type |
|---|---|---|---|
| P1 | holds → | R1 | Assignment edge (Resource → Process) |
| P1 | requests → | R2 | Request edge (Process → Resource) |
| P2 | holds → | R2 | Assignment edge (Resource → Process) |
| P2 | requests → | R1 | Request edge (Process → Resource) |

### Wait-for Graph

| Waiting Process | Waits For (Holder of needed resource) |
|---|---|
| P1 | P2 |
| P2 | P1 |

⚠ **Important:** A cycle in the wait-for graph with single-instance resources is both necessary and sufficient proof of deadlock — not just a risk indicator.

### Proposed Optimization

Impose a global, total ordering on resource acquisition (e.g. always acquire the lower-numbered resource first: R1 before R2) and make every process request resources in that same order. This changes only the synchronization/locking sequence — the critical section's actual work in P1 and P2 is untouched — and it structurally eliminates Circular Wait, since no process can ever hold R2 while waiting for R1.

### Corrected Pseudocode

```
Process P1:                    Process P2:  (only the lock ORDER changes)
  lock(R1)                       lock(R1)   ← was lock(R2)
  wait(100 ms)                   wait(100 ms)
  lock(R2)                       lock(R2)   ← was lock(R1)
  critical section               critical section
  unlock(R2)                     unlock(R2)
  unlock(R1)                     unlock(R1)
```

### Explanation of the Fix

With both processes now requesting R1 then R2, whichever process locks R1 first will always be able to subsequently acquire R2 once it's free (the other process cannot be holding R2 without first holding R1, which it can't get). No process can hold R2 while blocked on R1, so the cycle P1→P2→P1 can no longer form.

### Test Cases / Verification

### Best Practices to Prevent Recurrence

- Establish and document a global lock-ordering convention for every shared-resource pair in the codebase.
- Where ordering is impractical, use try-lock with a timeout and full rollback (release-and-retry) instead of blocking indefinitely.
- Run a periodic deadlock-detection pass (build the wait-for graph from held/requested locks and check for cycles) in long-running services as a safety net.

### Conclusion

The deadlock is caused by a classic circular-wait: opposite lock-acquisition order between two processes sharing two resources. Enforcing a single global lock order removes the circular wait without touching either process's actual logic, which is the least invasive and most standard fix for this exact pattern.

# QUESTION 2

*CPU Utilization Problem*

**10 MARKS**

## Original Question

**Q2. CPU Utilisation Problem**

A system shows low CPU utilisation (30%) even though processes are ready. You analyse the scheduler log and find frequent transitions.

Debug the cause using process management concepts and propose a system-level optimisation to improve utilisation.

*(10 Marks)*

## ANSWER

### Problem Statement

CPU utilization sits at just 30% despite the ready queue rarely being empty — find why the CPU is idle so often and propose a fix.

### Given Data / Assumptions

| Observation | Value / Note |
|---|---|
| Reported CPU utilization | 30% |
| Ready queue | Rarely empty (per problem statement) |
| Scheduler log pattern | Frequent Ready ↔ Waiting transitions with very short Running intervals (assumed, since not itemized in the source) |

📌 **Remember:** "Processes are ready" does not mean the CPU is busy — utilization depends on whether the OS can find a ready process to dispatch the instant the CPU frees up, not on queue length alone.

### Root Cause / Bug Identification

The “frequent transitions” are almost certainly Running→Waiting transitions happening very soon after dispatch — i.e. the workload is I/O-bound: each process runs briefly, then blocks for I/O almost immediately. If the degree of multiprogramming (number of processes in memory) is too low, there may be moments where every resident process is simultaneously blocked on I/O, leaving the CPU idle even though the (system-wide) ready queue is non-empty over the full log window.

### Theory — CPU Utilization vs. Multiprogramming

For n processes each I/O-waiting a fraction p of the time, CPU utilization ≈ $1 - p^n$ (assuming independence). A high p (I/O-bound mix) needs a larger n to keep utilization high; if n is too small for how I/O-heavy the workload is, utilization stays low no matter how efficient the CPU scheduler itself is.

### Process State Diagram

| | | |
|---|---|---|
| | **NEW** | |
| | ↓ (admitted) | |
| | **READY** | |
| | ↕ (dispatch / time-out) | |
| | **RUNNING** | |
| (I/O or event wait) ↙ | | ↘ (exit) |
| **WAITING** | | **TERMINATED** |
| (event completion → Ready) | | |

### Detailed Explanation

Every context switch away from a blocked process costs cycles and, more importantly, time — if the next dispatch has nothing ready to run, the CPU sits idle. The symptom (low utilization + processes technically "ready" over the full log, but not necessarily *simultaneously* ready) points at insufficient overlap between CPU-bound and I/O-bound work, not a scheduling-algorithm bug per se.

### Proposed Optimization

1. Increase the degree of multiprogramming (admit more resident processes) so that when one blocks for I/O, others are available to fill the CPU — directly raises the effective n in $U = 1 - p^n$.

2. Adopt (or re-tune) a scheduler that prioritizes I/O-bound processes immediately after they return from I/O (an MLFQ-style priority boost), so the CPU is re-filled with useful work the instant it frees, instead of waiting on a fixed time-slice rotation.

### Expected Improvement (Illustrative)

### Best Practices to Prevent Recurrence

- Monitor per-process I/O-wait fraction, not just aggregate CPU utilization, to catch this pattern early.
- Size the multiprogramming degree to the workload's I/O-wait profile rather than a fixed constant.
- Prefer asynchronous/non-blocking I/O in application code so a single process can overlap its own I/O with other useful work.

## Conclusion

Low utilization with a busy-looking ready queue is a classic sign of an I/O-bound workload outrunning the degree of multiprogramming. Raising multiprogramming and prioritizing I/O-returning processes both directly increase the fraction of time the CPU has something to do.

# QUESTION 3

*Memory Bottleneck Debugging*

**10 MARKS**

## Original Question

**Q3. Memory Bottleneck Debugging**

A multi-threaded program frequently encounters page faults, slowing performance. The memory map shows: Code: 5 MB, Heap: 150 MB, Stack: 1 MB per thread, 25 threads running, Physical RAM available: 2 GB.

Identify the memory issue causing performance degradation and propose two optimisations.

*(10 Marks)*

## ANSWER

### Problem Statement

A 25-thread program with a 150 MB heap suffers frequent page faults on a machine with 2 GB of RAM — identify the memory issue and propose two fixes.

### Given Data

### Root Cause / Bug Identification

One instance (≈180 MB) comfortably fits in 2 GB on paper, so the fault rate is not explained by a single instance alone — the realistic causes are: (a) several instances/copies of this process (or other memory-heavy processes) run concurrently, pushing total resident memory close to or over the 2 GB physical limit, forcing continuous eviction and refetching of heap pages; and/or (b) the 150 MB heap has poor locality (large, sparsely-accessed data structures), so even without over-commitment, the *working set* the threads actively touch exceeds the frames the OS is willing to keep resident for this process, causing thrashing at the process level.

### Theory — Working Set & Locality

The working set is the set of pages a process references in its most recent Δ time units; performance requires the OS to keep that whole set resident.

Working Principle: If allocated frames < working-set size, nearly every reference outside the resident subset faults, and the fault rate spikes independent of total available RAM elsewhere on the machine.

Advantages: Directly explains fault rate without needing to guess at other processes.

Disadvantages: Requires runtime reference tracking to size correctly.

Applications: Memory-management tuning in JVM/OS-level allocators.

### Proposed Optimizations

1. Reduce and bound the per-thread footprint: cap the thread pool size (e.g. via a fixed worker pool) instead of running 25 full-stack OS threads for the same work, and/or reduce individual stack size where 1 MB is more than the call depth actually needs — this shrinks the 25 MB stack overhead proportionally.

2. Improve heap locality and sizing: profile the 150 MB heap for sparse/rarely-touched allocations, use memory pooling or object reuse to shrink the active working set, and/or increase available physical RAM (or reduce the number of concurrent instances) so the working set fits comfortably within resident frames.

### Expected Improvement (Illustrative)

### Best Practices to Prevent Recurrence

- Track working-set size in production monitoring, not just total heap allocated.
- Prefer bounded thread pools over unbounded thread-per-task models.
- Right-size stack allocations instead of relying on the language/runtime default for every thread.

### Conclusion

The fault rate points to the active working set (not necessarily the raw 180 MB footprint) exceeding resident frames, likely worsened by concurrent instances or sparse heap access. Bounding thread count and improving heap locality both shrink the working set directly.

# QUESTION 4

*Starvation Debugging*

**10 MARKS**

## Original Question

**Q4. Starvation Debugging**

You observe that process P5 never gets CPU time in a multilevel queue scheduler: Q1 (Highest Priority): Interactive, Q2: System, Q3: Batch, Time Quantum: Q1 = 5ms, Q2 = 10ms, Q3 = FCFS. P5 is in Q3 but keeps getting postponed.

Debug the root cause of starvation and suggest an optimisation to ensure fairness.

*(10 Marks)*

## ANSWER

### Problem Statement

P5 sits in the lowest-priority batch queue (Q3) of a fixed-priority multilevel queue scheduler and never runs — identify why and fix it.

### Given Data

### Root Cause / Bug Identification

This is a fixed-priority Multilevel Queue (MLQ): a lower queue is only serviced when every higher queue is completely empty. As long as Q1 or Q2 ever has a ready process — which is essentially guaranteed under continuous interactive/system load — Q3 never gets dispatched at all. There is no mechanism in plain MLQ to guarantee Q3 any CPU time, so P5 experiences indefinite (unbounded) starvation, not just delay.

### Theory — MLQ vs. Aging

Multilevel Queue scheduling has no built-in fairness guarantee across queues by design — it assumes the process's queue assignment reflects a genuinely fixed priority for its whole lifetime. The standard fix for the starvation this causes is aging: gradually increase a waiting process's effective priority the longer it waits, until it eventually gets promoted (or guaranteed a CPU slice) regardless of new arrivals in higher queues.

### MLQ Structure with Starvation Point

| **Q1 — Interactive (quantum 5ms, serviced first, always)** |
|---|
| ↓ |

| Q2 — System (quantum 10ms, serviced only if Q1 empty) |
|---|
| ↓ |
| **Q3 — Batch/FCFS — P5 here (⚠ serviced only if Q1 AND Q2 both empty — rarely true)** |

### Proposed Optimization

Convert the fixed MLQ into a Multilevel Feedback Queue (MLFQ) with aging: track each process's waiting time in Q3, and after a threshold (e.g. 500 ms without running), temporarily promote it to Q2 (or grant it a guaranteed minimum CPU percentage per scheduling cycle, e.g. "Q3 gets at least 10% of CPU time every 1 second regardless of Q1/Q2 load"). Either mechanism bounds P5's maximum wait time.

### Expected Improvement (Illustrative)

### Best Practices to Prevent Recurrence

- Never deploy a pure fixed-priority MLQ for mixed interactive+batch workloads without an aging or guaranteed-share mechanism.
- Log per-process maximum wait time in monitoring, not just average, to catch starvation that averages might hide.
- Prefer MLFQ over static MLQ when workload composition (interactive vs. batch mix) is not tightly controlled.

### Conclusion

P5's starvation is the expected behaviour of a strict fixed-priority MLQ under sustained higher-queue load, not a bug in any single component. Aging or a guaranteed minimum CPU share for Q3 both directly bound P5's worst-case wait.

# QUESTION 5

*Context-Switch Overhead Debugging*

**10 MARKS**

## Original Question

**Q5. Context-Switch Overhead Debugging**

A server performs 20,000 context switches/sec, causing high overhead. Log details show: Average burst time per thread: 2ms, I/O wait frequency: very high, Preemptive scheduler with quantum = 1ms.

Explain why context switching is so high and propose two optimisations to reduce overhead.

*(10 Marks)*

## ANSWER

### Problem Statement

20,000 context switches/sec is causing high overhead on a server whose threads average a 2 ms burst under a 1 ms preemptive quantum — explain the cause and propose two fixes.

### Given Data

### Root Cause / Bug Identification

The quantum (1 ms) is smaller than the average burst (2 ms), so a typical thread is forcibly preempted mid-burst at least once before it would have blocked or finished naturally — that alone roughly doubles the minimum number of switches versus a quantum sized to the burst. On top of that, the "very high" I/O wait frequency means threads are also frequently switched out involuntarily on top of the timer-driven preemptions, and switched back in again on I/O completion — compounding the count further.

### Theory — Quantum Sizing

A quantum much smaller than the average burst maximizes fairness/responsiveness granularity but at a direct, linear cost in context-switch overhead, since more switches are needed to complete the same total work. The classic RR trade-off is: quantum too large → degrades toward FCFS-like response times; quantum too small → overhead dominates useful work. 1 ms vs. a 2 ms average burst is on the "too small" side of that trade-off for this workload.

### Detailed Explanation (Illustrative Switch Count)

### Proposed Optimizations

1. Increase the time quantum to be closer to (or slightly above) the observed average burst time (e.g. 2–3 ms instead of 1 ms), so most threads complete or naturally block within a single slice instead of being preempted mid-burst — this is the single highest-leverage change here.

2. Adopt an MLFQ-style scheduler that gives I/O-bound threads a short quantum (for responsiveness) but avoids penalizing them with a full preemption cycle every time they return from I/O — e.g. priority boost on I/O completion instead of re-queuing at the back with a fresh timer-driven preemption soon after.

### Best Practices to Prevent Recurrence

- Periodically compare quantum size against measured average burst time as workload characteristics drift.
- Track context-switch rate as a first-class metric alongside CPU utilization, not just after overhead becomes visible to users.
- Prefer adaptive/feedback-based quantum sizing over a single static constant for mixed workloads.

### Conclusion

The excessive switch rate is explained by a quantum undersized relative to the average burst, compounded by frequent I/O-driven switches. Right-sizing the quantum to the workload's actual burst profile is the direct fix, with MLFQ-style I/O handling as a complementary refinement.

# QUESTION 6

*Semaphore Synchronization Debugging*

**10 MARKS**

## Original Question

**Q6. Semaphore Synchronization Debugging**

A producer-consumer application occasionally hangs during execution. Semaphore values observed: mutex = 1, empty = 0, full = 10, Buffer size = 10.

Producer: wait(mutex) → wait(empty) → produce item → signal(full) → signal(mutex)

Consumer: wait(mutex) → wait(full) → consume item → signal(empty) → signal(mutex)

Identify the synchronization bug causing the system to hang. Explain which deadlock condition is being violated. Suggest an optimization to ensure correct synchronization without changing the producer-consumer functionality.

*(10 Marks)*

## ANSWER

### Problem Statement

A classic producer-consumer implementation hangs intermittently — find the exact ordering bug in the given semaphore code and fix it without changing what produce/consume actually do.

### Given (Buggy) Code

```
Producer:                        Consumer:
  wait(mutex)                      wait(mutex)
  wait(empty)                      wait(full)
  produce item                     consume item
  signal(full)                     signal(empty)
  signal(mutex)                    signal(mutex)
```

### Bug Identification

Both Producer and Consumer acquire the binary mutex FIRST, before checking the counting semaphore (empty/full) that tells them whether they actually can proceed. When the buffer is full (empty = 0), the Producer blocks on wait(empty) — but it is still holding mutex at that point, since signal(mutex) never ran. The Consumer, which needs mutex to consume an item and free up a slot (signal(empty)), can never acquire mutex because the Producer is stuck holding it. Neither side can make progress — the system hangs.

### Which Deadlock Condition Is Violated

This is a Hold-and-Wait → Circular Wait deadlock between the two semaphores: the Producer holds mutex while waiting on empty, and the Consumer (which would eventually signal empty) cannot get

past its own wait(mutex) to do so. The two semaphores form a two-party circular dependency: Producer holds mutex, waits for empty — which only Consumer can supply — while Consumer needs mutex, which only Producer can release.

### Detailed Trace (Buffer Full Scenario)

### Proposed Optimization

Swap the acquisition order in both Producer and Consumer so the counting semaphore (empty/full) is checked BEFORE the mutex is taken. This way, a process only ever holds the mutex for the brief duration of the actual buffer access, never while blocked waiting on buffer space/items — the functionality (what produce/consume do) is completely unchanged; only the wait() ordering changes.

### Corrected Code

```
Producer:                          Consumer:
  wait(empty)    ← swapped           wait(full)    ← swapped
  wait(mutex)    ← swapped           wait(mutex)   ← swapped
  produce item                       consume item
  signal(mutex)                      signal(mutex)
  signal(full)                       signal(empty)
```

### Test Cases / Verification

### Best Practices to Prevent Recurrence

- Standing rule: acquire counting semaphores (resource-availability signals) before the mutual-exclusion semaphore, and release mutex before signaling the counting semaphore, if possible — mutex should be the innermost lock.
- Prefer higher-level constructs (monitors with condition variables, or language-level concurrent queues) over hand-written semaphore ordering wherever available, since this exact ordering mistake is easy to make by hand.
- Add a hang-detection timeout in test suites specifically exercising the empty-buffer and full-buffer boundary conditions.

### Conclusion

The hang is caused by acquiring the mutex before the counting semaphore in both Producer and Consumer, creating a circular hold-and-wait exactly at the buffer-full/empty boundary. Reordering the two wait() calls (counting semaphore first, mutex second) removes the circular dependency while leaving produce/consume functionality identical.

# QUESTION 7

*Thrashing Detection & Optimization*

**10 MARKS**

## Original Question

**Q7. Thrashing Detection and Optimization**

A virtual memory system reports: Number of processes: 50, Average page fault rate: 250 faults/sec, CPU Utilization: 18%, Disk Utilization: 95%, Free Frames Available: 2%. The operating system suspects thrashing.

Debug the root cause of the performance degradation. Explain the relationship between page faults and CPU utilization. Propose two operating system optimizations to reduce thrashing.

*(10 Marks)*

## ANSWER

### Problem Statement

50 concurrent processes, near-zero free frames, 95% disk utilization, and only 18% CPU utilization — confirm and explain thrashing, then propose two fixes.

### Given Data

### Root Cause / Debugging the Degradation

This is textbook thrashing: the degree of multiprogramming (50 processes) has been pushed past the point where physical memory can hold each process's working set. With only 2% of frames free, nearly every memory reference outside a tiny resident set triggers a page fault; the OS spends almost all its time servicing page faults (95% disk utilization) rather than running processes (18% CPU). Adding more processes at this point makes CPU utilization worse, not better, because each new process only steals frames from the others, increasing everyone's fault rate.

### Relationship Between Page Faults and CPU Utilization

As the degree of multiprogramming increases from a low starting point, CPU utilization initially rises (more work available to fill idle CPU time). But once resident memory can no longer hold every active process's working set, page-fault rate rises sharply, and CPU utilization collapses because most time goes to swapping pages in/out (disk-bound) instead of executing instructions. This produces the classic "thrashing curve" — utilization peaks, then falls off a cliff as multiprogramming keeps increasing. The reported 250 faults/sec + 18% CPU + 95% disk sits well past that peak, on the falling side of the curve.

### Illustrative Thrashing Curve

### Proposed Optimizations

1. Reduce the degree of multiprogramming directly: have the medium-term scheduler suspend (swap out) a subset of processes until free frames recover to a healthy margin, guided by each process's working-set size rather than a fixed process count.

2. Adopt working-set-based or Page-Fault-Frequency (PFF) admission control and local (per-process) page replacement instead of pure global replacement, so a memory-hungry process cannot starve others of frames, and new processes are only admitted when enough frames are free to support their working set.

### Best Practices to Prevent Recurrence

- Monitor free-frame percentage and page-fault rate together as an early-warning pair, not CPU utilization alone (which can look merely “low” rather than obviously wrong).
- Cap admission of new processes based on a working-set/PFF budget, not just a static process-count limit.
- Prefer local replacement policies in workloads with highly variable per-process memory needs.

### Conclusion

All four reported metrics are consistent with a system well past its optimal multiprogramming point: memory is exhausted, the disk is saturated servicing faults, and the CPU is starved as a direct consequence. Reducing multiprogramming and moving to working-set-aware admission/replacement both attack the actual over-commitment rather than any single symptom.

# QUESTION 8

*File System Performance Debugging*

**10 MARKS**

## Original Question

**Q8. File System Performance Debugging**

A database server experiences slow file access times. System logs indicate: Average file size: 500 MB, Disk seeks per request: 120, Disk utilization: 92%, Cache hit ratio: 15%.

Identify the file system bottleneck affecting performance. Explain why disk access latency is increasing. Suggest two optimizations to improve file access efficiency.

*(10 Marks)*

## ANSWER

### Problem Statement

A database server with large (500 MB average) files is slow to access, with 120 disk seeks per request, 92% disk utilization, and only a 15% cache hit ratio — identify the bottleneck and propose two fixes.

### Given Data

### Root Cause / Bottleneck Identification

120 seeks for a single large-file request is far more than a well-laid-out sequential read should need, which points to file fragmentation — the file's blocks are scattered non-contiguously across the disk (likely from a linked or poorly-managed indexed allocation scheme without defragmentation), forcing the disk head to jump repeatedly instead of streaming. This is compounded by the very low 15% cache hit ratio, meaning the buffer cache is too small (or poorly tuned) to keep frequently-accessed blocks resident, so most reads pay full seek+rotation latency instead of being served from memory.

### Why Disk Access Latency Is Increasing

Latency = seek time + rotational delay + transfer time, and seek time dominates when access is non-sequential. With 120 seeks per request, the disk head is doing 120 expensive repositioning operations instead of one continuous read — each seek can cost orders of magnitude more time than the data transfer itself. The low cache hit ratio means this expensive path is taken on 85% of accesses instead of being absorbed by memory, so the aggregate latency the database observes is dominated by disk-seek overhead rather than actual data throughput.

### Proposed Optimizations

1. Improve file layout: use extent-based or contiguous allocation for large files (or run periodic defragmentation/reallocation) so a 500 MB file is stored as few, large contiguous extents instead of many scattered blocks — directly cuts the seeks-per-request from 120 toward close to 1.

2. Improve caching: increase buffer cache size and/or adopt read-ahead/prefetching tuned for large sequential files, and consider a cache-replacement policy (e.g. LRU-K or ARC) that resists eviction of frequently re-scanned database pages — directly raises the 15% hit ratio.

### Expected Improvement (Illustrative)

### Best Practices to Prevent Recurrence

- Track seeks-per-request and cache hit ratio as standing file-system health metrics, not just after users report slowness.
- Schedule periodic defragmentation/reallocation for workloads with large, frequently-modified files.
- Size buffer cache and prefetch windows to the actual working set of hot files, and revisit as data volume grows.

### Conclusion

The bottleneck is disk seek overhead from a fragmented large-file layout, worsened by an undersized/poorly-tuned cache that lets 85% of accesses fall through to that expensive path. Fixing file layout and cache sizing together addresses both the cause and the amplifier.

# QUESTION 9

*Thread Scheduling Debugging*

**10 MARKS**

## Original Question

**Q9. Thread Scheduling Debugging**

A web server creates one thread per client request. During peak hours: Active Threads: 5,000, CPU Utilization: 60%, Memory Usage: 92%, Average Response Time: 12 seconds. Scheduler logs show frequent thread creation and termination.

Identify the primary issue affecting server performance. Explain how thread management contributes to the delay. Recommend two optimizations to improve scalability and response time.

*(10 Marks)*

## ANSWER

### Problem Statement

A thread-per-request web server hits 5,000 active threads at peak, with only 60% CPU utilization but 92% memory usage and 12-second response times — identify the primary issue and propose two fixes.

### Given Data

### Root Cause / Primary Issue

The thread-per-request model creates and tears down an OS thread for every single client connection. At 5,000 concurrent requests this means 5,000 kernel-scheduled threads competing for the run queue, each carrying its own stack (memory overhead, hence 92% usage) and kernel scheduling structures. CPU utilization is only 60% — not because there isn't enough work, but because so much time and memory is spent on thread creation/destruction and context-switch/scheduling overhead among 5,000 entities that useful request-handling work is starved of both CPU cycles and memory.

### How Thread Management Contributes to the Delay

- Creation/termination overhead: spinning up and tearing down a full OS thread per request (visible in the "frequent creation and termination" logs) adds fixed latency to every single request, multiplied by request volume at peak.
- Memory pressure: 5,000 thread stacks plus kernel-level thread bookkeeping consume the 92% memory headroom, increasing the chance of paging/swapping, which independently adds latency on top of scheduling overhead.

- Scheduler contention: the OS scheduler itself must manage run-queue operations across 5,000 entities, and context-switching among that many threads adds overhead that doesn't correspond to any actual request-handling progress — explaining why CPU utilization looks moderate (60%) while response time is still terrible.

### Proposed Optimizations

1. Replace thread-per-request with a bounded thread pool: a fixed (or auto-scaled within a cap) number of worker threads pulling requests from a queue, eliminating per-request thread creation/termination overhead entirely and capping memory usage regardless of request volume.

2. Move to an event-driven / asynchronous I/O model (reactor pattern) or an M:N (user-level-to-kernel) threading model, so a small number of kernel threads can multiplex thousands of concurrent connections — directly removing the 1:1 thread-to-request coupling that is the root cause.

### Expected Improvement (Illustrative)

### Best Practices to Prevent Recurrence

- Never let concurrency model scale 1:1 with client connections in a public-facing server; always bound worker concurrency explicitly.
- Monitor thread-creation rate as a first-class metric, since high churn is itself a leading indicator before memory/response-time symptoms appear.
- Load-test with realistic peak concurrency before production to catch this class of bottleneck pre-emptively.

### Conclusion

The primary issue is the unbounded thread-per-request concurrency model itself, not a scheduling algorithm bug — 5,000 real OS threads impose creation, memory, and context-switch overhead that starves useful work. A bounded thread pool or async/event-driven model removes the 1:1 coupling and is the standard fix at this scale.

# QUESTION 10

*Disk Scheduling Optimization*

**10 MARKS**

## Original Question

**Q10. Disk Scheduling Optimization Problem**

A disk scheduling log shows the following request queue: 98, 183, 37, 122, 14, 124, 65, 67. Current Head Position: 53. The system uses FCFS scheduling and reports: Average seek time: Very High, Throughput: Low.

Debug why FCFS is causing poor performance in this scenario. Compare FCFS with one suitable disk scheduling algorithm for this workload. Propose an optimization to improve seek time and overall throughput.

*(10 Marks)*

## ANSWER

### Problem Statement

FCFS disk scheduling for request queue [98, 183, 37, 122, 14, 124, 65, 67] from head position 53 is producing very high seek time and low throughput — quantify why, compare against an alternative, and propose a fix.

### Given Data

### Root Cause — Why FCFS Performs Poorly Here

FCFS services requests strictly in arrival order with no regard for their physical proximity to the current head position or to each other. The given sequence swings the head back and forth across the disk repeatedly (53→98→183→37→122→14→124→65→67) instead of sweeping in one direction — each large jump costs a full seek, and there is no logic in FCFS to reorder these for locality.

### Calculation — FCFS Total Seek Distance

### Comparison — SSTF (Shortest Seek Time First)

| Step | Move | Distance |
|---|---|---|
| 1 | 53 → 65 | 12 |
| 2 | 65 → 67 | 2 |
| 3 | 67 → 37 | 30 |
| 4 | 37 → 14 | 23 |

| Step | Move | Distance |
|---|---|---|
| 5 | 14 → 98 | 84 |
| 6 | 98 → 122 | 24 |
| 7 | 122 → 124 | 2 |
| 8 | 124 → 183 | 59 |
| Total | | 236 cylinders |

💡 **Key Concept:** SSTF always picks the physically nearest pending request, minimizing each individual seek — but it can starve far-away requests indefinitely under continuous heavy load, since new nearby requests keep arriving and jumping the queue.

### Comparison — LOOK (practical SCAN, for reference)

| Step | Move | Distance |
|---|---|---|
| 1 | 53 → 65 | 12 |
| 2 | 65 → 67 | 2 |
| 3 | 67 → 98 | 31 |
| 4 | 98 → 122 | 24 |
| 5 | 122 → 124 | 2 |
| 6 | 124 → 183 | 59 |
| 7 | 183 → 37 | 146 |
| 8 | 37 → 14 | 23 |
| Total | | 299 cylinders |

LOOK sweeps in one direction to the last pending request, then reverses — no starvation risk, at a modest seek-distance cost compared to SSTF.

### Summary Comparison Table

### Proposed Optimization

Replace FCFS with SSTF for this specific batch (lowest total seek distance, 236 vs. 640 cylinders — a 63% reduction), since a finite, already-known request set has no ongoing-arrival starvation risk. For a live, continuously-arriving production workload where starvation is a real concern, adopt LOOK (or

C-LOOK for more uniform wait times) instead — still a 53% seek reduction over FCFS, with a bounded worst-case wait that SSTF cannot guarantee.

### Best Practices to Prevent Recurrence

- Never default to FCFS for disk scheduling once request volume/spread is non-trivial — it has no locality awareness at all.
- Choose SSTF only for bounded/batch workloads; prefer SCAN/C-SCAN/LOOK for continuously-arriving production traffic to avoid starvation.
- Monitor total seek distance and per-request wait-time variance, not just average seek time, to catch starvation that an average can mask.

### Conclusion

FCFS's 640-cylinder total seek comes from ignoring locality entirely, evidenced directly by the back-and-forth request order. SSTF cuts this to 236 (best for this batch), while LOOK offers a strong 299-cylinder result with no starvation risk — the right production choice is LOOK/C-LOOK, with SSTF reserved for bounded, non-recurring batches.

Rubrics for Calculation

| **Criteria** | **Weightage** | **Level 4 (Exemplary)** | **Level 3 (Proficient)** | **Level 2 (Developing)** | **Level 1 (Beginning)** |
|---|---|---|---|---|---|
| Problem Identification | 20% | Accurately identifies all errors and root causes with clear understanding | Identifies most errors with minor gaps | Identifies some errors but misses key issues | Unable to identify errors correctly |
| Debugging | 25% | Applies systematic debugging techniques effectively to resolve issues | Uses appropriate debugging methods with minor errors | Limited debugging approach; requires guidance | Unable to debug or resolve issues |
| Optimization | 20% | Optimizes code by significantly improving time complexity using efficient algorithms | Improves time complexity with minor gaps in efficiency | Limited improvement in time complexity | No improvement; inefficient approach retained |
| Analysis | 20% | Demonstrates strong logical reasoning and clear justification of solutions | Shows reasonable analysis with some justification | Limited analysis; weak reasoning | No analytical reasoning demonstrated |
| Code Quality | 15% | Produces clean, well-structured code with proper comments and documentation | Code is mostly clear with minor documentation gaps | Code lacks structure and proper documentation | Poorly written code with no documentation |